

Introduction to Git & Github?



git

Git



What is Git?

- **Git** is a version control system.
- **Git** helps you keep track of code changes.
- **Git** is used to collaborate on code.

Git



What does Git do? 🤔

- Manage projects with **Repositories**
- **Clone** a project to work on a local copy
- Control and track changes with **Staging** and **Committing**
- **Branch** and **Merge** to allow for work on different parts and versions of a project
- **Pull** the latest version of the project to a local copy
- **Push** local updates to the main project

Using Git with Command Line?



Git version?

```
git --version  
git version 2.40.0.windows.1
```

Configure Git?

```
git config --global user.name "your_github_name"  
git config --global user.email "your_github_email@gmail.com"
```

Creating Git Folder?

```
mkdir myproject  
cd myproject
```

Creating Git Folder?

```
git init  
Initialized empty Git repository in /Users/user/myproject/.git/
```

Git

Git Adding New Files?

Create index.html?

`touch index.html`

Confirm the file?

ls

`git status`

Git Log?

`git log`



Git



Git Staging Environment?

As you are working, you may be adding, editing and removing files. But whenever you hit a milestone or finish a part of the work, you should add the files to a Staging Environment

Staged files are files that are ready to be **committed**

Stage a single file?

```
git add index.html  
git status
```

Stage many files?

```
git add -all OR git add . OR git add -A
```

Git

Git Commit?

Since we have finished our work, we are ready move from **stage** to **commit** for our repo. When we **commit**, we should always include a **message**

```
git commit -m "First release of Hello World!"
```



Git Commit without Stage?

Note: Short status flags are:

- ?? - Untracked files
- A - Files added to stage
- M - Modified files
- D - Deleted files

Git Help?

- Git help --all
- Git help add

Example?

```
git commit -a -m "Updated index.html with a new line"
```

Warning: Skipping the Staging Environment is not generally recommended.

Git

Git Branch?

In Git, a branch is a new/separate version of the main repository

New Git Branch?

```
git branch hello-world-images
```

List all Branches?

```
git branch
```

Switching to existing Branches?

```
git checkout hello-world-images
```

Switching and Creating a new Branch?

```
git checkout -b emergency-fix
```



Git

Branch Merge?

We have the emergency fix ready, and so let's merge the master and emergency-fix branches.

Switch to Your main[**master**]?

`git checkout master`

Merge with hello-world-images ?

`git merge hello-world-images`

Delete the Merged branch?

`git branch -d hello-world-images`

Confirm available Branches?

`git branch`



Git

Creating a Merge Conflict?



Merge Conflict occurs when Git cannot automatically resolve differences between two commits during a merge process

We need two branches master and hello-world-images which are all in sync
Make changes to master, and commit. Then make changes to hello-world-images

Switch to master and merge. [🚧 🚧 🚧 🚧 🚧]

Switch to Your master?

git checkout master

Merge with hello-world-images ?

git merge hello-world-images

Resolve the changes on index.html file then?

git add index.html

git status

git commit -m "merged with hello-world-images after fixing conflicts"

😳😳 → → → → GitHub Incoming → → → → 😏😏😏



What is GitHub?

- Git is not the same as GitHub.
- GitHub makes tools that use Git.
- GitHub is the largest host of source code in the world, and has been owned by Microsoft since 2018.

[Pull requests](#)[Issues](#)[Marketplace](#)[Explore](#)

Repositories

[New](#)

Working with a team?

GitHub is built for collaboration. Set up an organization to improve the way your team works together, and get access to more features.

[Create an organization](#)

Discover interesting projects and people to populate your news feed.

Your news feed helps you keep up with recent activity on repositories you [watch](#) and people you [follow](#).

[Explore GitHub](#)

ProTip! The feed shows you events from people you [follow](#) and repositories you [watch](#).



Subscribe to your news feed

[New repository](#)[Import repository](#)[New gist](#)[New organization](#)[New project](#)

Create a Repository on GitHub

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Repository template

No template ▾

Start your repository with a template repository's contents.

Owner *

Repository name *

 deniswachira ▾

 /

Great repository names are short and memorable. Need inspiration? How about [reimagined-octo-enigma](#) ?

Description (optional)

☒  **Public**

Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**

You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: **None** ▾

Push Local Repository to GitHub

Quick setup — if you've done this kind of thing before

 Set up in Desktop

or

HTTPS

SSH

<https://github.com/w3schools-test/hello-world.git>



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

- Copy the URL, or click the clipboard marked in the image above.
- Now paste it the following command:

```
git remote add origin https://github.com/your-username/gitrepo.git
```

GitHub



Connect our Local Repository to GitHub

Push to github

```
git push --set-upstream origin master
```

Pull from GitHub

pull is a combination of 2 different commands:

- Fetch - gets all the change history of a tracked branch/repo.
- Merge - combines the current branch, with a specified branch

```
git pull origin - update our local Git:
```

Push Changes to GitHub

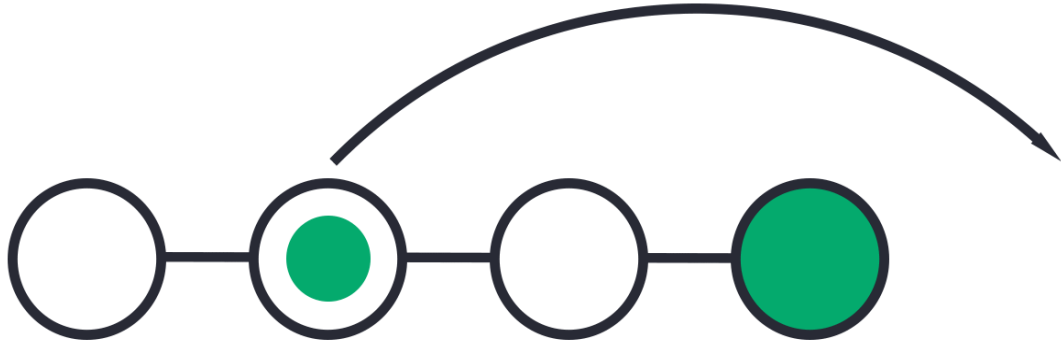
```
git commit -a -m "Updated index.html for push to github"
```


Git Undo

Git Revert

revert is the command we use when we want to take a previous commit and add it as a new commit, keeping the log intact

Step 1: Find the previous commit:



Step 2: Use it to make a new commit:



Git Revert

Find commit in Log

```
git log --oneline
```

Git Revert HEAD

This command will create a new commit that undoes the changes from the most recent commit.

```
git revert abc123
```

This command will create a new commit that undoes the changes from the commit with the hash "abc123". Replace "abc123" with the actual hash of the commit you want to revert.

```
git revert HEAD~2..HEAD
```

This command will create a new commit that undoes the changes from the two most recent commits.

GIT REBASE

Git rebase is a command that allows developers to integrate changes from one branch to another.

Git rebase compresses all the changes into a single "patch." Then it integrates the patch onto the target branch. Unlike merging, rebasing flattens history. It transfers the completed work from one branch to another. In the process, unwanted history is eliminated.

Git Rebase vs. Merge Similarities

Git rebase and merge both integrate changes from one branch into another. Where they differ is how it's done. Git rebase moves a feature branch into a master. Git merge adds a new commit, preserving the history

Git



Git Rebase vs. Merge : Differences

Git Rebase

Streamlines a potentially complex history.

Avoids merge commit "noise" in busy repos with busy branches.

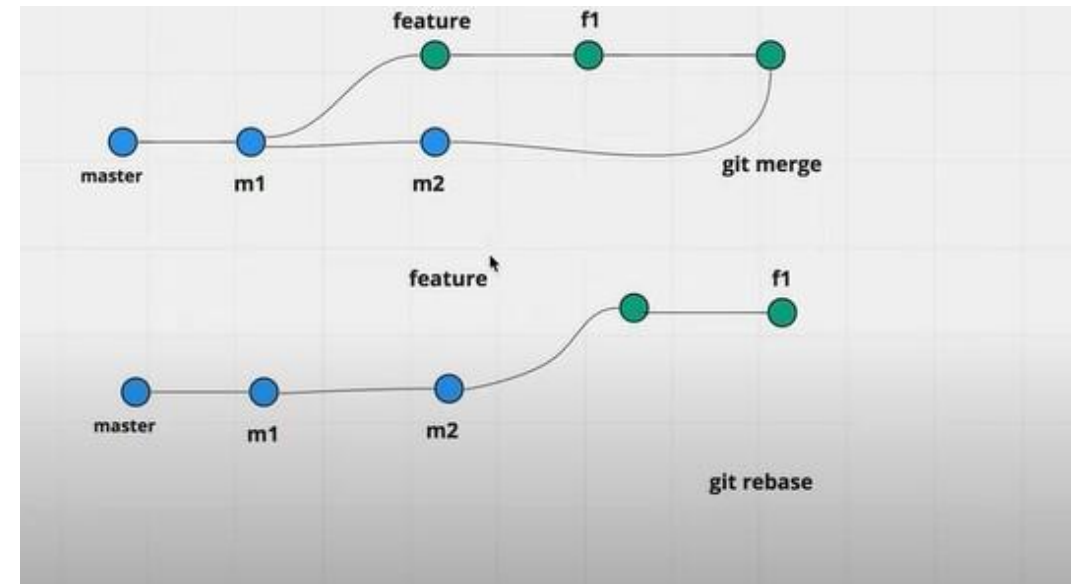
Cleans intermediate commits by making them a single commit, which can be helpful for DevOps teams.

Git Merge

Simple and familiar.

Preserves complete history and chronological order.

Maintains the context of the branch.



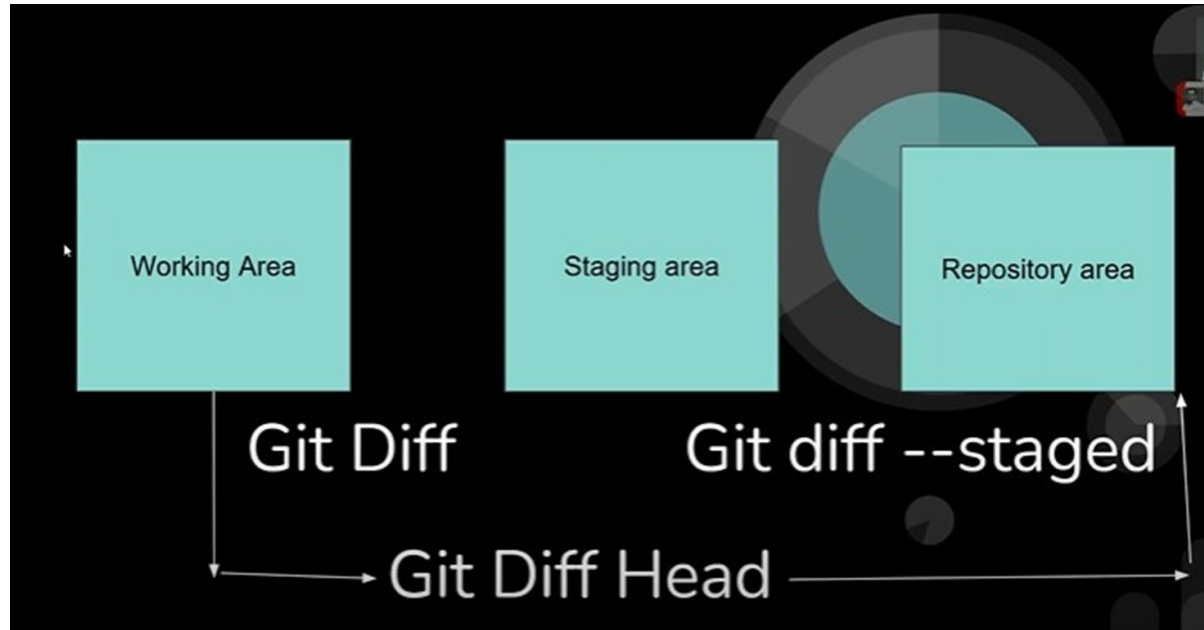
Git

Git diff

git diff command is used in git to track the difference between the changes made on a file. Since Git is a version control system tracking changes are something very vital to it. Diff command takes two inputs and reflects the differences between them. It is not necessary that these inputs are files only. It can be branches, working trees, commits, and more.



Changes that have made and are not yet committed
What changes are not staged



Git diff commits

Comparing files between two different commits

```
git diff 7514eeb2a2713dac16dc27ad6a71aea92f2bb4f0  
cb9e6e409a50028416957e4d0ec9a40dffec6224
```

Git diff branches

```
Git diff branch1 branch2
```

```
git diff
git diff --cached    (git commit)
git diff HEAD        (git commit -a)
git diff -w
```

```
git diff <commit>
git diff --cached <commit>
git diff <commit> <commit>
```

```
git diff feature main
git diff feature...main
git diff feature main file.txt
```

- ◀ Changed and not staged for commit?
- ◀ Changed and staged for commit?
- ◀ Changed since last commit?
- ◀ Ignore whitespace differences

- ◀ Specific commit and current?
- ◀ Specific commit and staged?
- ◀ Difference between two commits?

- ◀ Difference between tips of branches?
- ◀ Changed in main since feature was started off of it
- ◀ Difference in file.txt on two branches?

Git Squash

You can merge several commits into a single commit with the compelling interactive rebase command.

```
git log --oneline  
  
git merge-base ticket1 main  
  
git rebase -i <commit-sha>  
  
pick 1285e6c starting f1  
squash b714e68 more work in f1  
squash 985d1e1 completed f1  
  
# This is the 1st commit message:  
  
# This is the commit message #2:  
  
# This is the commit message #3:  
completed feature f1
```

- ◀ See the branch history
- ◀ Get the original base of the “ticket1” branch created from main
- ◀ Start the rebase from the commit sha
- ◀ Squash the commits you want to combine into a single commit
- ◀ Choose the commit message you want to use.

Git



Git Cherry Pick

Git cherry-pick is a powerful command that enables arbitrary Git commits to be picked by reference and appended to the current working HEAD.

Cherry picking is the act of picking a commit from a branch and applying it to another.

Syntax

```
git cherry-pick <commitSha>
```

When to use git Cherry Pick

- Accidentally make a commit in the wrong branch.
- Bug hotfixes
- Team collaboration.

Git

Git Cherry Pick



```
git log --oneline
```

```
git log <branch-name> --oneline
```

```
git checkout <branch-name>
```

```
git cherry-pick <commit-sha>
```

◀ Find the commit you want

◀ List the log from another branch

◀ Where do you want to put the commit?
Checkout that branch.

◀ Perform the cherry-pick to append the
specified commit to HEAD

Assignment 1 : Git GitHub Flow

The GitHub flow works like this:

- Create a github repo
- fork
- Open a Pull Request
- Make changes and add Commits
- Review
- Deploy
- Merge

In groups of 8 Students, think of a simple project, one student to create a github repo, the rest folk, create a pull-up request, using eight Branches contribute to that project. Do not delete the branches. A project can be a simple website.

Assignment 2 : Git Ignore and .gitignore

When sharing your code with others, there are often files or parts of your project, you do not want to share.

Examples

- log files
- temporary files
- hidden files
- personal files

create a github repo, name it gitignore cheatsheet, create an gitignore_cheatesheet.md file. Inside the file list rules for matching patters in .gitignore files. Example

Pattern	Explanation/Matches	Examples
name.file	All files with the name.file will be ignored	/name.file /lib/name.file

END

GitHub



END